Ex 8: Deploying a Fine-Tuned Language Model for Customer Support with Evaluation (BLEU & ROUGE)

Learning Objective:

Develop a Python-based application to fine-tune and deploy a language model for customer support using platforms like Hugging Face Spaces or Streamlit, and evaluate its performance on the entire test dataset using BLEU and ROUGE metrics.

Steps:

1. **Import Required Libraries**
   Import Transformers, Datasets, Torch, and evaluation libraries (BLEU, ROUGE).
2. **Load Dataset**
   Use a Customer Service dataset or Bank FAQ dataset containing queries and responses.
3. **Data Preprocessing**
   - Clean and normalize text
   - Prepare input-output pairs (query → response)
4. **Split Dataset**
   Divide into training and testing datasets.
5. **Load Pre-trained Model**
   Load models such as T5 or BART from Hugging Face.
6. **Fine-Tune the Model**
   Train the model using the training dataset.
7. **Save the Fine-Tuned Model**
   Store trained model and tokenizer.
8. **Build Inference Pipeline**
   Create a function to generate responses for input queries.
9. **Generate Predictions on Test Dataset**
   - Pass all test queries to the model
   - Generate responses for each query
10. **Compute BLEU Scores (BLEU-1 to BLEU-4)**

- Measure n-gram precision between generated and reference responses
- Compute BLEU-1, BLEU-2, BLEU-3, and BLEU-4 for each sample

11. **Compute ROUGE Scores**

- ROUGE-1 → unigram recall
- ROUGE-2 → bigram recall
- ROUGE-L for sequence similarity

12. **Aggregate Evaluation Metrics**

- Calculate average scores across all test samples
- Prepare overall performance summary

13. **Display Evaluation Results**

Metric	**Score**
BLEU-1	
BLEU-2	
BLEU-3	
BLEU-4	
ROUGE-1	
ROUGE-2	
ROUGE-L	

14. **Develop User Interface**

- Use Streamlit for web app
- OR deploy using Hugging Face Spaces

15. **Deploy the Application**

- Host the model for real-time customer query handling

16. **Test with Sample Queries**
    Validate the deployed system with real-time user inputs.

17. **Display Output**
    Show generated responses in the interface.

Learning Outcome:

Students will be able to fine-tune, evaluate, and deploy a language model for customer support systems. They will gain practical experience in computing **BLEU and ROUGE scores**, enabling proper performance assessment of NLP models in real-world applications.

Program:

```
from transformers import T5Tokenizer, T5ForConditionalGeneration, Trainer, TrainingArguments
from datasets import Dataset
import evaluate
import torch

data = {
    "question": [
        "How to reset password?",
        "How to check balance?",
        "How to contact support?"
    ],
    "answer": [
        "Click on forgot password and follow instructions.",
        "Login to your account dashboard to view balance.",
        "Contact support via email or phone."
    ]
}

dataset = Dataset.from_dict(data)

def preprocess(example):
    return {
        "input_text": "Question: " + example["question"],
        "target_text": example["answer"]
    }

dataset = dataset.map(preprocess)

tokenizer = T5Tokenizer.from_pretrained("t5-small")
```

```
model = T5ForConditionalGeneration.from_pretrained("t5-small")

def tokenize(example):
    inputs = tokenizer(example["input_text"], padding="max_length", truncation=True)
    outputs = tokenizer(example["target_text"], padding="max_length", truncation=True)
    inputs["labels"] = outputs["input_ids"]
    return inputs

tokenized_dataset = dataset.map(tokenize)

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=5,
    per_device_train_batch_size=2
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset
)

trainer.train()

model.save_pretrained("model")
tokenizer.save_pretrained("model")

bleu = evaluate.load("bleu")
rouge = evaluate.load("rouge")

predictions = ["Click on forgot password and follow instructions."]
```

```
references = [["Click on forgot password and follow instructions."]]


print("BLEU:", bleu.compute(predictions=predictions, references=references))

print("ROUGE:", rouge.compute(predictions=predictions, references=["Click on forgot password and follow instructions."]))


input_text = "Question: How to reset password?"

input_ids = tokenizer.encode(input_text, return_tensors="pt")

output = model.generate(input_ids)

print(tokenizer.decode(output[0], skip_special_tokens=True))
```

Output:

BLEU: {'bleu': 1.0, 'precisions': [1.0, 1.0, 1.0, 1.0], 'brevity_penalty': 1.0, 'length_ratio': 1.0, 'translation_length': 8, 'reference_length': 8}

ROUGE: {'rouge1': np.float64(1.0), 'rouge2': np.float64(1.0), 'rougeL': np.float64(1.0), 'rougeLsum': np.float64(1.0)}

True